Obstacles to Parallel Programming

Yun Tian (Tony)
Department of Computer Science
EWU

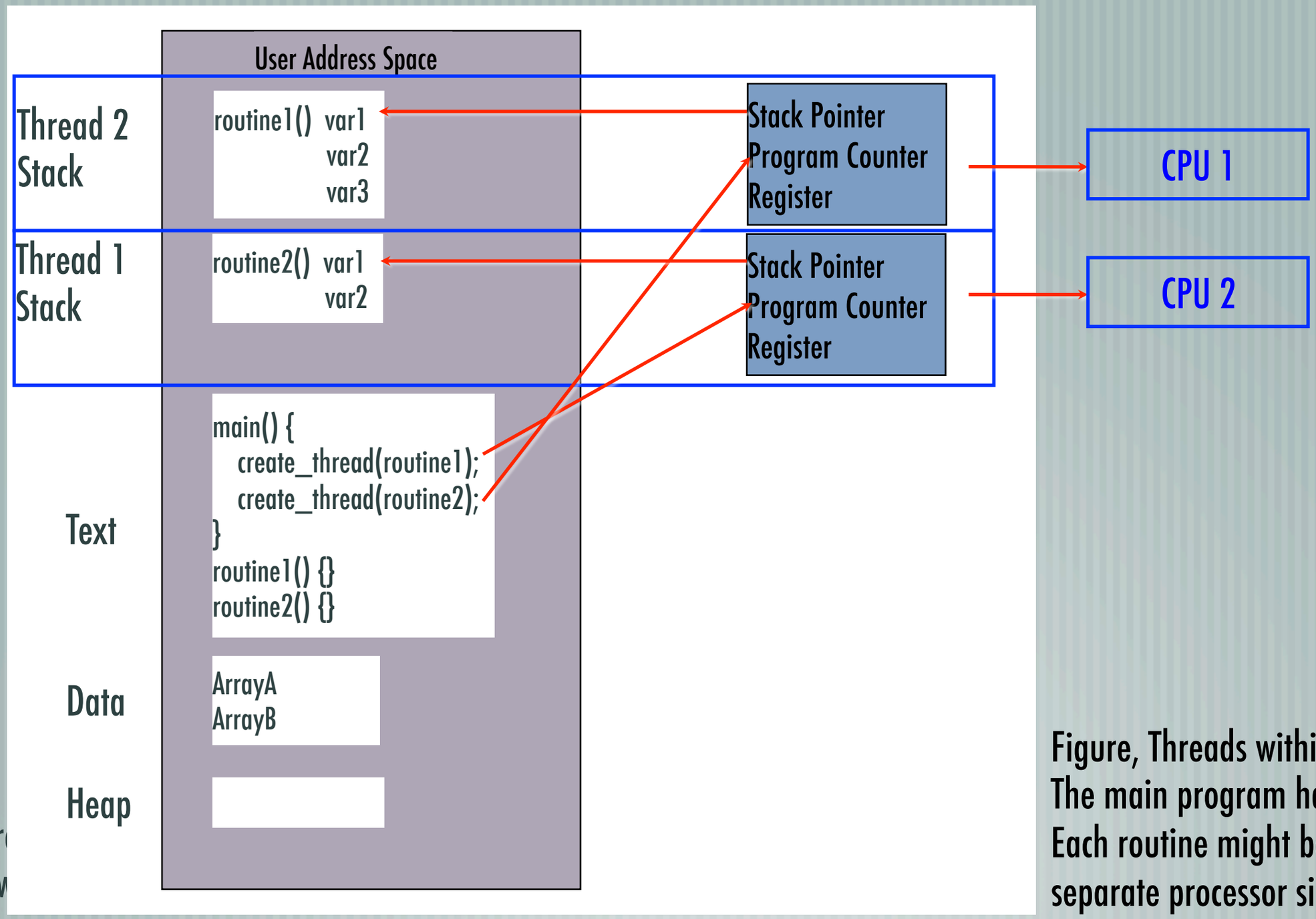
Outline

- Concept of a thread
- Four solutions to counting 3s problem
- Conclusion

Concept of A Thread

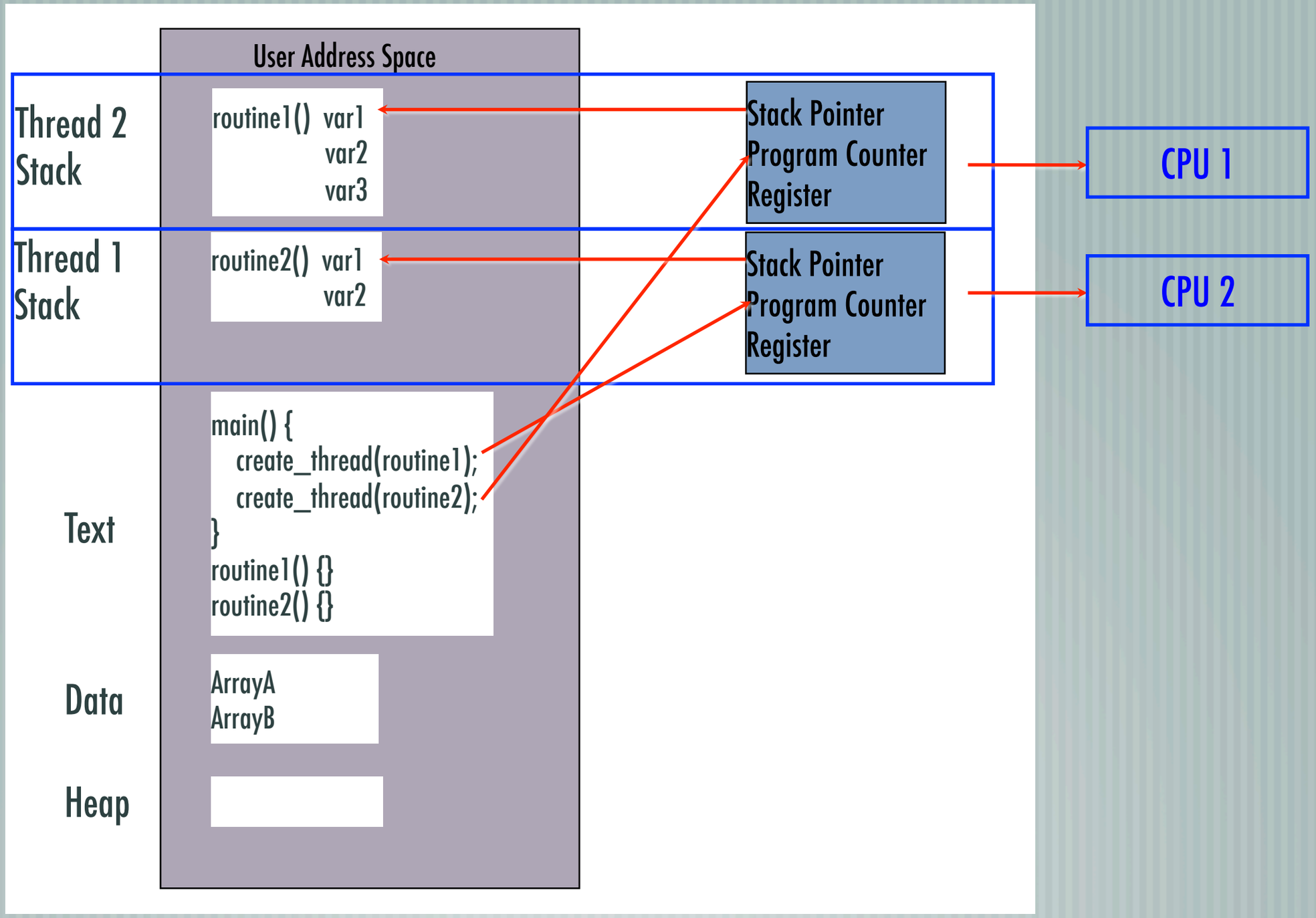
- Thread: independent stream of instructions that can be scheduled to run by the operating system.
  - A thread has everything needed to execute a stream of instructions.
    - Private program text, a call stack and a program counter
  - All threads belonging to the same process (program) can see the memory belonging to the program.
    - Multiple threads can cooperate to compute on global data.

Concept of A Thread

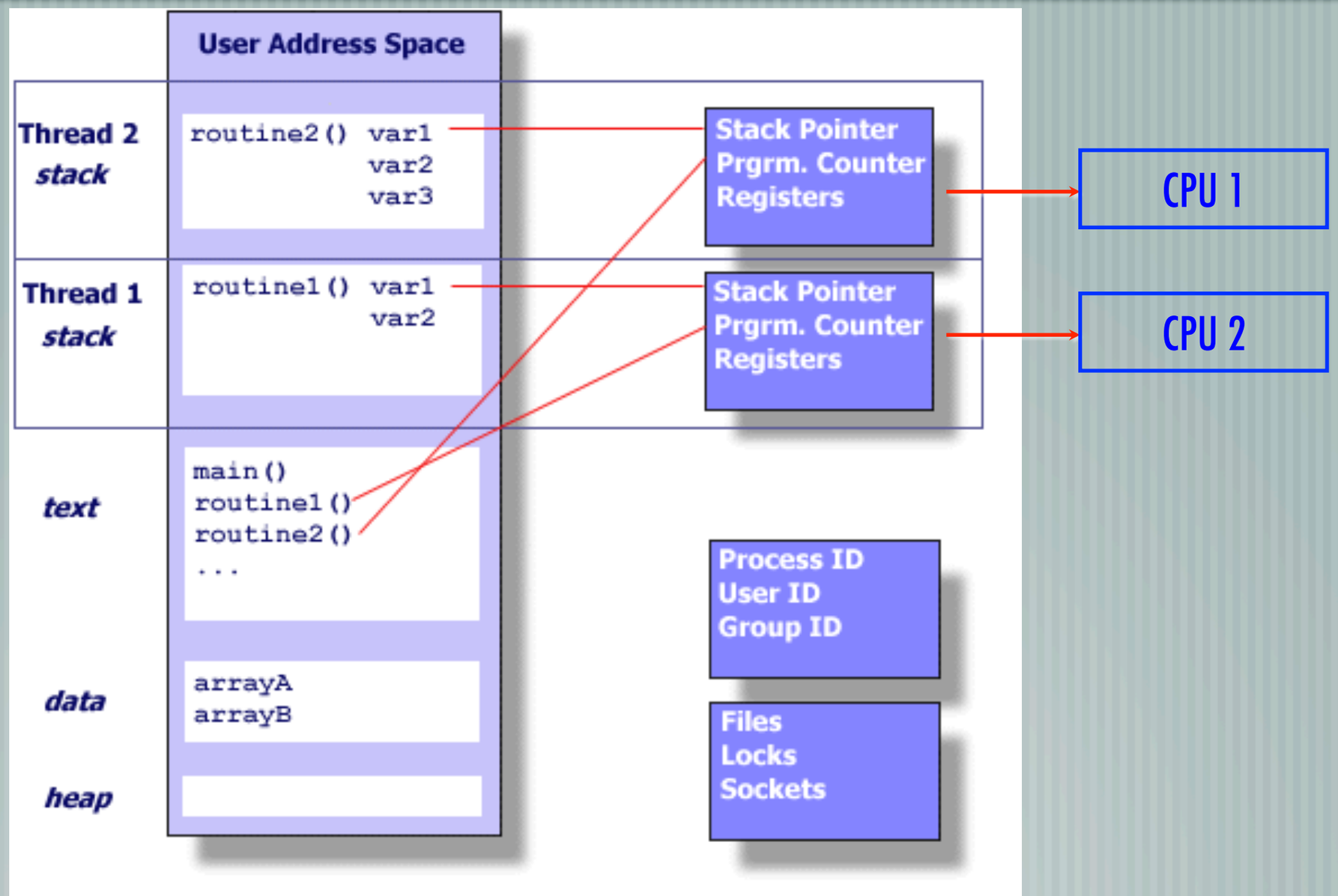


Figure, Threads within a Unix Process. The main program has two routines. Each routine might be running on a separate processor simultaneously.

Figur
follov


User Address Space
Thread 2 Stack
routine1() var1
var2
var3
Thread 1 Stack
routine2() var1
var2
Text
main() {
create_thread(routine1);
create_thread(routine2);
}
routine1() {}
routine2() {}
Data
ArrayA
ArrayB
Heap
Stack Pointer
Program Counter
Register
Stack Pointer
Program Counter
Register
CPU 1
CPU 2

Concept of A Thread



this looks like serial code.
Perhaps add a

run_thread(routine1)

Figure, Threads within a Unix Process. The main program has two routines. Each routine might be running on a separate processor simultaneously. (Picture from https://computing.llnl.gov/tutorials/pthreads/#Thread)

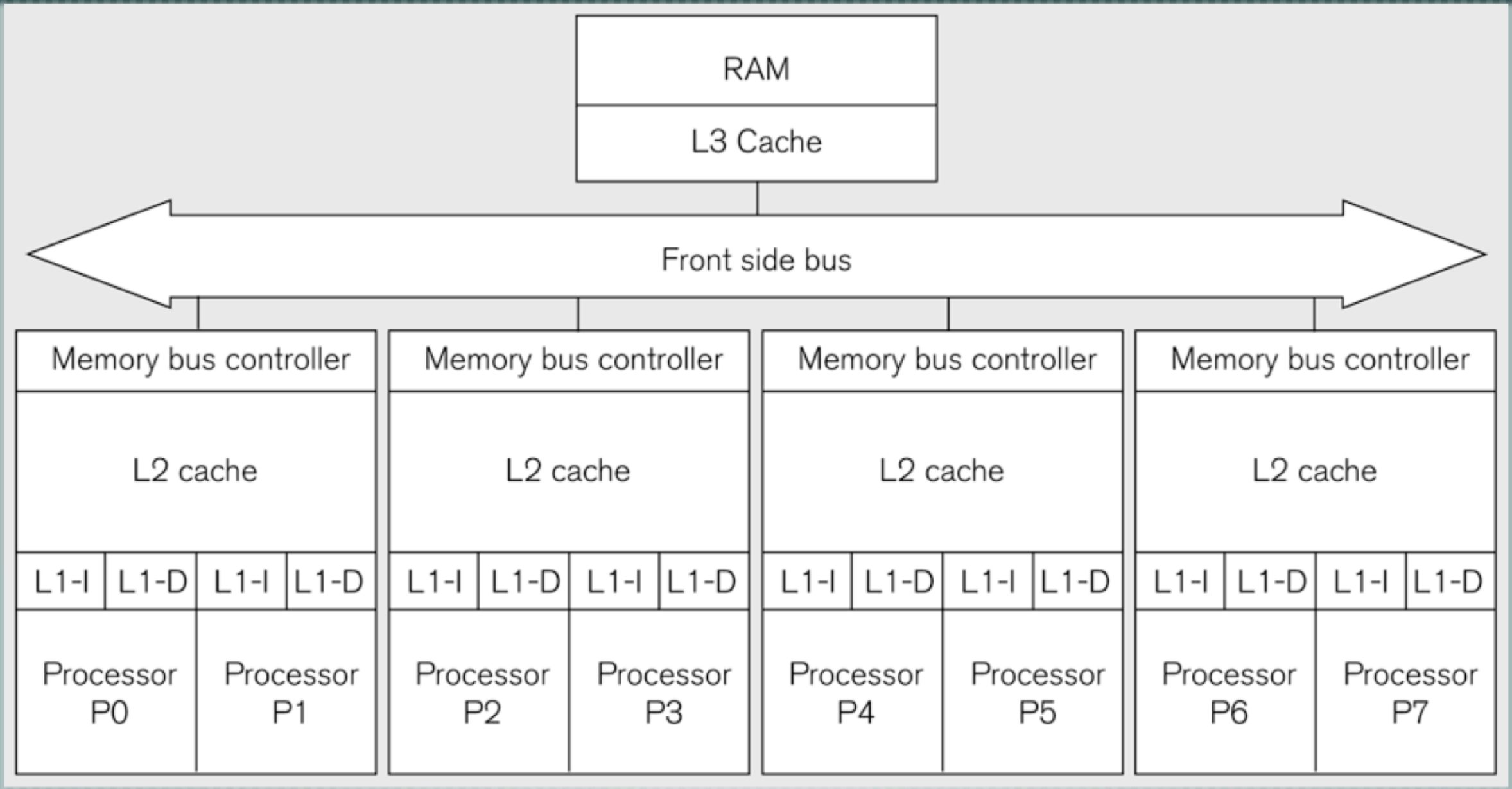
Concept of A Thread

- A thread or a thread of execution is a unit of parallelism.
  - On multiple CPUs, threads may execute truly simultaneously.
  - A program begins with a single thread, the main thread. We can create arbitrary number of threads using functions.
  - When a thread finishes, it typically join the main thread.

Organization of Multi-core

- Because the underlying hardware affects performance, we first look at an Intel multi-core CPU.
- This is a Intel Xeon 7100 dual core chips consisting of 8 processors.

Organization of Multi-core



Figure, Organization of a multi-core computer system on which the experiments are run. Each processor has a private L1 cache; it shares an L2 cache with its "chip-mate" and shares an L3 cache with the other processors.

Organization of Multi-core

- When the processor needs to read from (or write to) RAM, it first checks whether a copy of that data is in the cache.
  - Read from or write to cache is much quicker than reading from or writing to RAM.
- Data is transferred between memory and cache in blocks of fixed size, called cache lines.
  - Changing one byte in a cache line make the whole cache line stale.

Counting 3s Problem

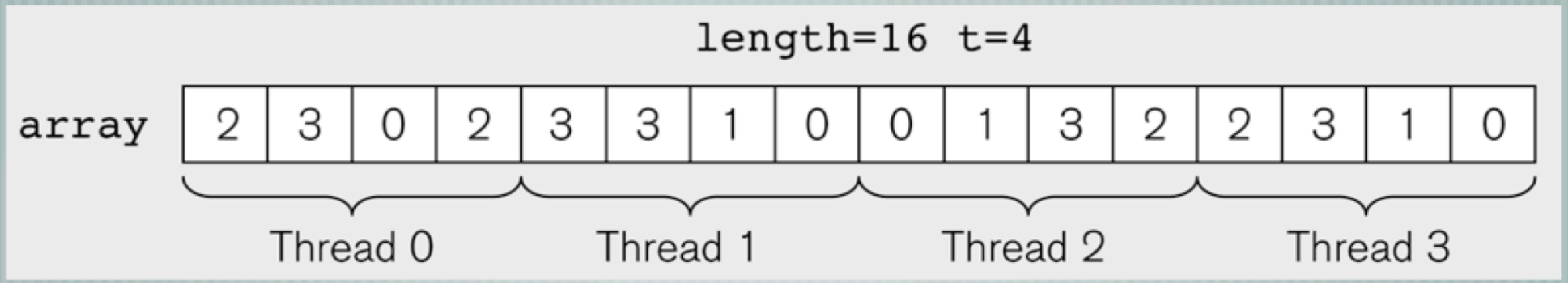
Serial code for counting 3s problem

```
int *array;
int length;
int count;

int count3s()
{
    int i;
    count=0;
    for(i = 0; i < length; i ++)
    {
        if(array[i] == 3)
            count ++;
    }
    return count;
}
```

Multithreaded Solution

- Multithreaded solution to counting 3s
  - We have to partition the array.
  - Each thread only processes (n / t) array elements, where t is the number of threads and n is the length of the array.



Figure, Data allocation to threads. Allocations are consecutive indices.

Multithreaded solution one to counting 3s

```
int t;                    /* number of threads */
int *array;
int length;
int count;

void count3s()
{
  int i;
  count = 0;
  /* Create t threads */
  for(i=0; i<t; i++)
  {
    thread_create(count3s_thread, i);
  }

  return count;
}

void count3s_thread(int id)
{
  /* Compute portion of the array that this thread
     should work on */
     int length_per_thread=length/t;
     int start=id*length_per_thread;

  for(i=start; i<start+length_per_thread; i++)
  {
    if(array[i]==3)
    {
      count++;
    }
  }
}
```

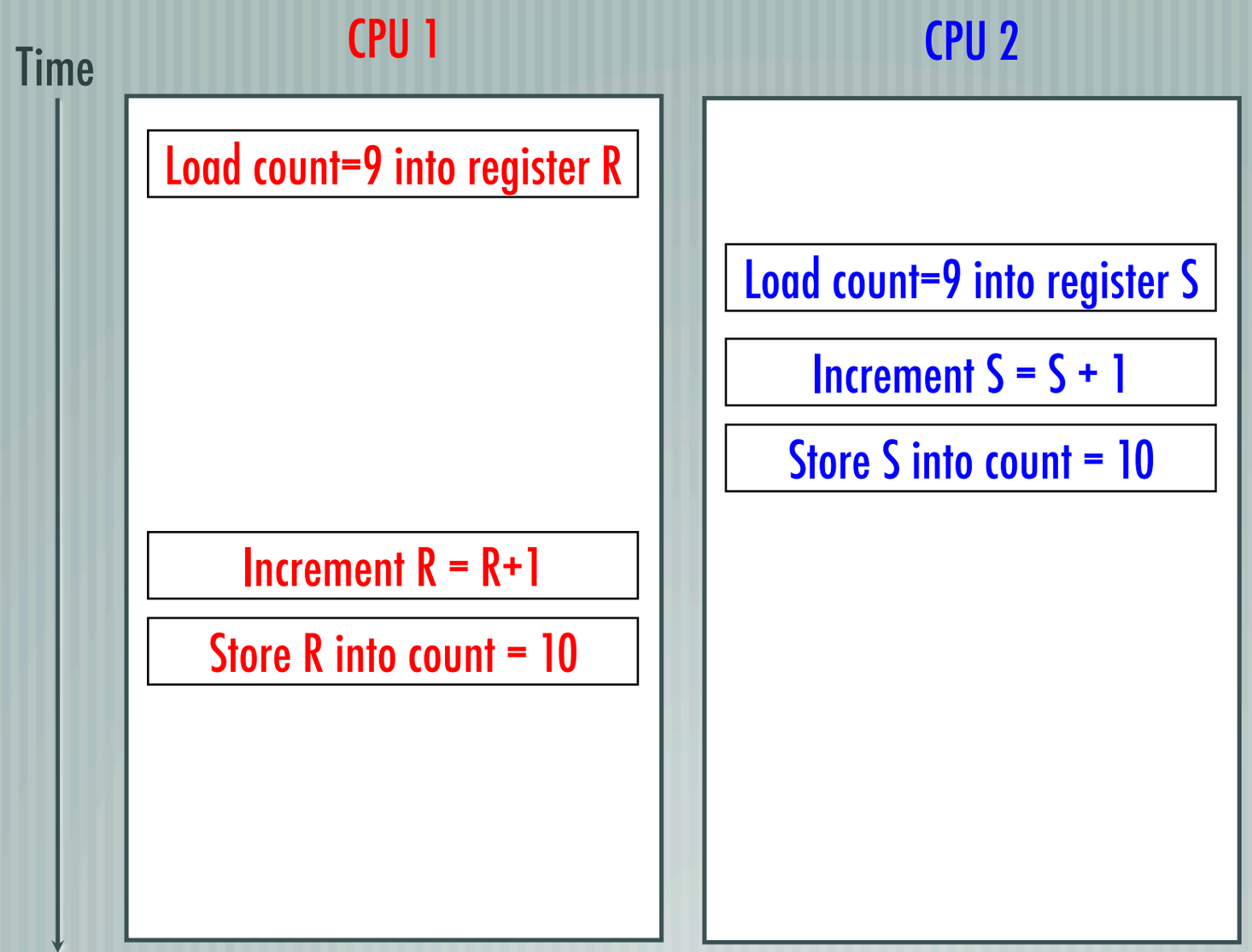
Multithreaded Solution One

- Solution one does not produce correct answer. Why?
  - There is race condition when incrementing count variable.
- Race condition: the result of an execution depends on the timing of two or more events.
  - Occurs when two or more threads write to the same unprotected variable.
  - Timing depends on system load and scheduling policy.

Multithreaded Solution One

- Why does a race condition occur?
  - The statement `count ++` is implemented as a series of simple operations:
    - Load count into a register
    - Increment count
    - Store count back to memory
  - These operations could be interleaved on different CPUs.

Multithreaded Solution One



Figure, a bad interleaving causes lost update.

Multithreaded Solution Two

- Solution two uses a mutex
  - We use a mutex to ensure that only one thread executes a certain section of code at a time (mutual exclusion).
    - This section of code is called a critical section.
  - A mutex is an object that has two states: locked and unlocked.
    - We have two operations lock() and unlock() to change its state.

Multithreaded Solution Two

- Two operations on a mutex

```
mutex_lock(mutex) {
    if mutex is already locked by other thread
            calling thread blocks / sleep until mutex is unlocked
    mutex = locked
}
```

```
mutex_unlock(mutex) {    if mutex is locked by this thread
if a thread w is waiting on this mutex
give the mutex to w
wake thread w
else            mutex = unlocked    else            error}
```

Multithreaded Solution Two

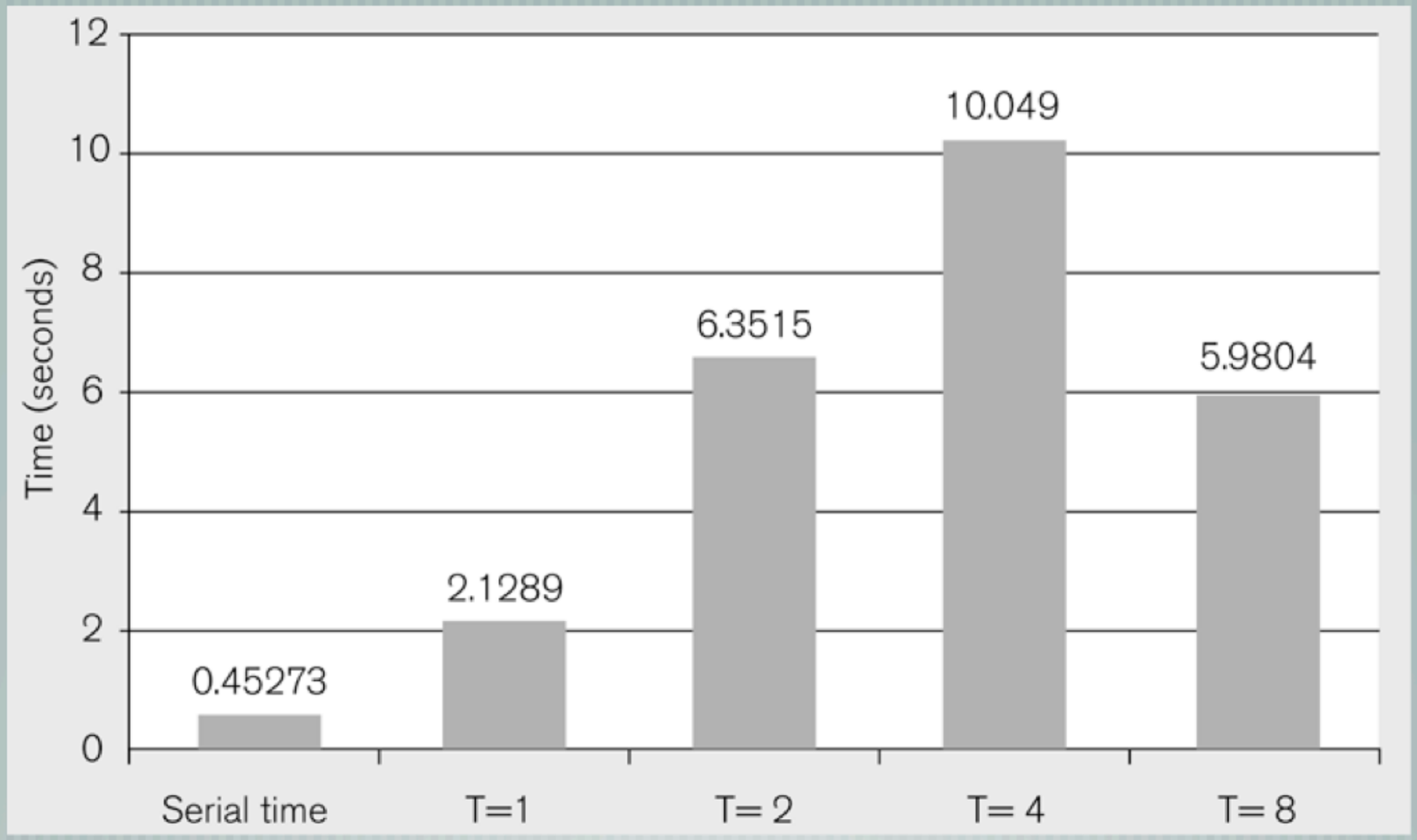
```
mutex m;

void count3s_thread(int id)
{
  /* Compute portion of the array that this thread
     should work on */
  int length_per_thread=length/t;
  int start=id*length_per_thread;

  for(i=start; i<start+length_per_thread; i++)
  {
    if(array[i]==3)
    {
      mutex_lock(m);
      count++;
      mutex_unlock(m);
    }
  }
}
```

Figure, The second try at a Count 3s solution showing the count3s_thread() with mutex protection for the count variable.

Multithreaded Solution Two



Figure, performance of our second Count 3s solution. Performance becomes worse as we increase number of threads.

Multithreaded Solution Two

- Why does performance degrade as we add more processors?
  - The use of mutex hurts the performance.
  - Lock contention degrades performance as each thread spends additional time waiting for the critical section to be unlocked.
    - We access the critical section too frequently.
    - For each integer 3 in array, threads have to compete for the mutex.

Multithreaded Solution Two

- Improve Solution Two
  - We reduce the frequency of overall count access.
    - When find a integer 3, each thread increment a private_count.
    - Accumulating the local contribution in the private_count to the overall count.
  - We only access the critical section for updating count once per thread.

Multithreaded Solution Three

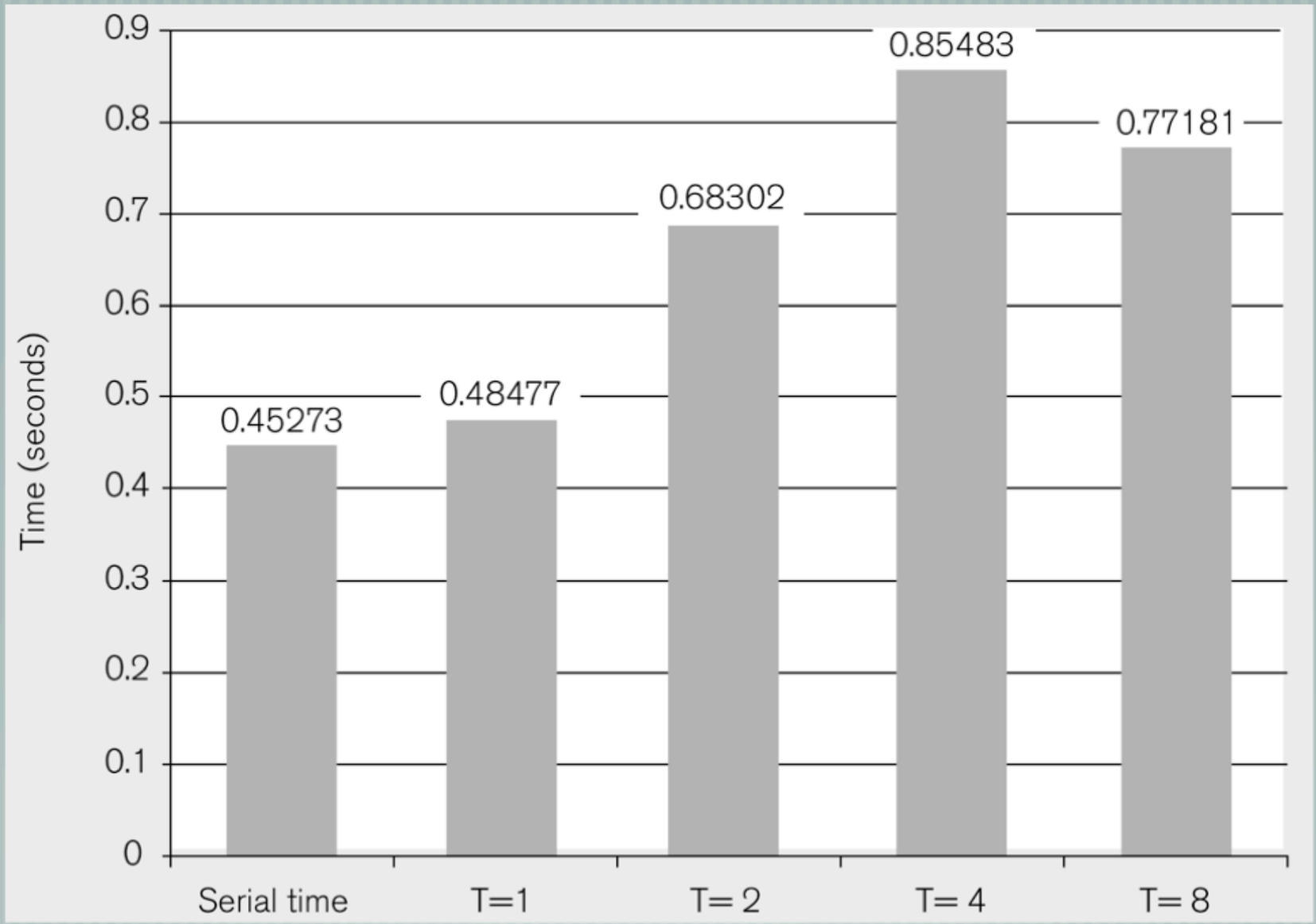
```
private_count[MaxThreads];
mutex m;

void count3s_thread(int id)
{
  /* Compute portion of array for this thread to
     work on */
  int length_per_thread=length/t;
  int start=id*length_per_thread;

  for(i=start; i<start+length_per_thread; i++)
  {
    if(array[i] == 3)
    {
      private_count[id]++;
    }
  }
  mutex_lock(m);
  count+=private_count[id];
  mutex_unlock(m);
}
```

Figure, the count3s_thread() for our third Count 3s solution using private_count array elements.

Multithreaded Solution Three



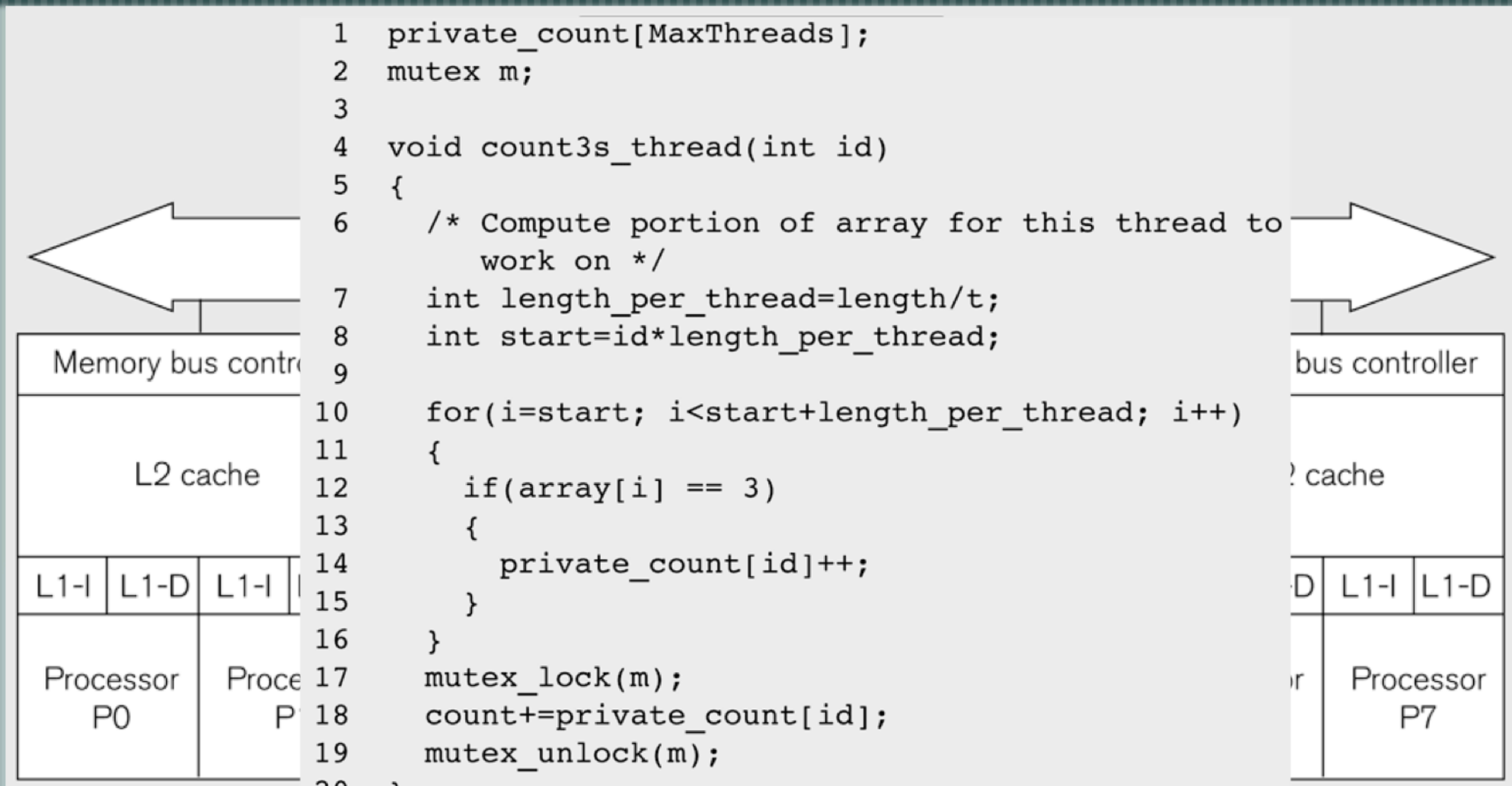
Figure, performance of our third Count 3s solution. Performance is improved when compared with solution 2. but performance degrades as we increase number of threads.

- define cache line? Show memory divided into lines?
- How much slower is RAM compared to cache?

Multithreaded Solution Three

- Why does performance degrade as we add more processors?
  - Hardware uses a protocol to maintain coherent caches.
  - If processor 0 modifies a value in a cache line, the hardware will invalidate any cached copy of that block that resides in processor 1's L1 cache.
  - Cache coherence protocol becomes costly if two processors take turns repeatedly modifying the same data.

Organization of Multi-core




```
private_count[MaxThreads];
mutex m;

void count3s_thread(int id)
{
  /* Compute portion of array for this thread to
     work on */
  int length_per_thread=length/t;
  int start=id*length_per_thread;

  for(i=start; i<start+length_per_thread; i++)
  {
    if(array[i] == 3)
    {
      private_count[id]++;
    }
  }
  mutex_lock(m);
  count+=private_count[id];
  mutex_unlock(m);
}
```

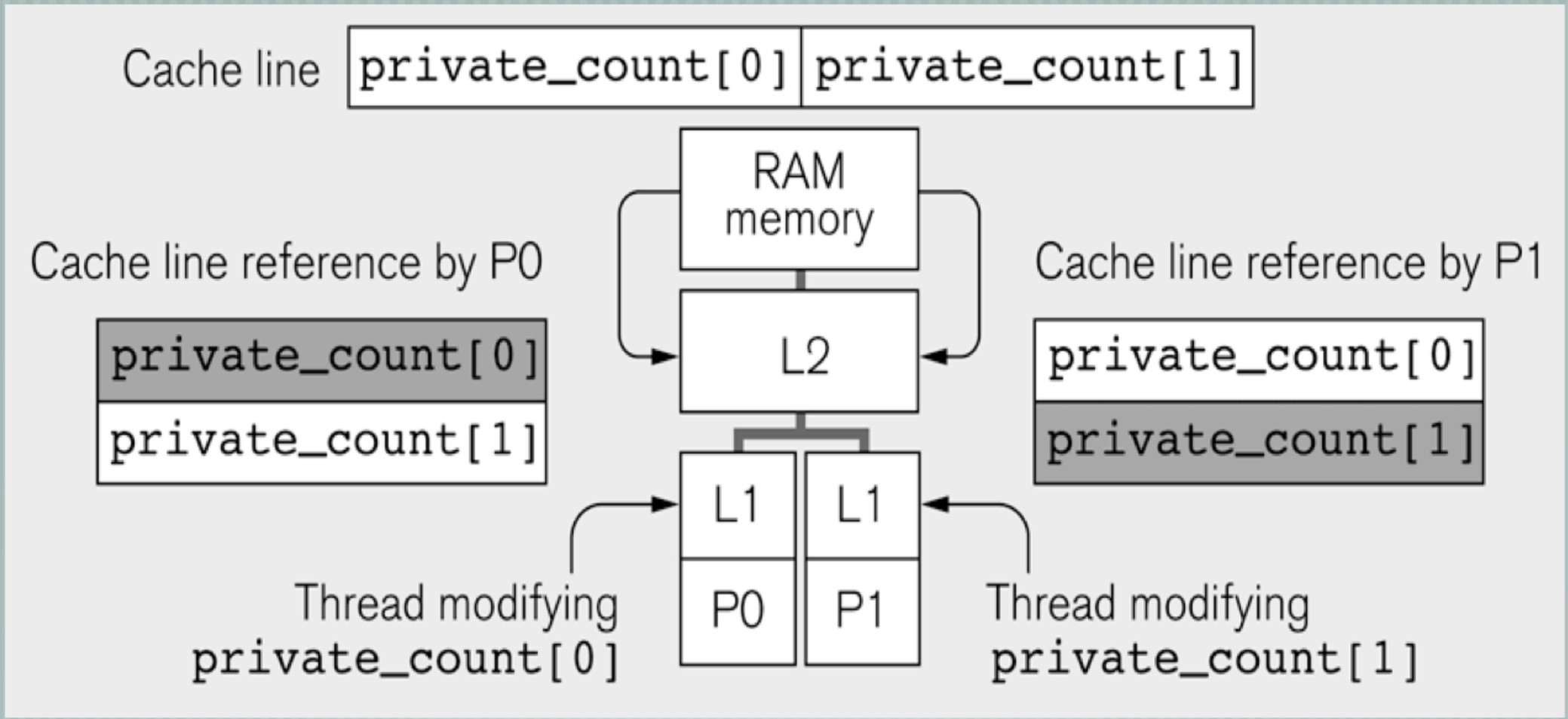
Figure, The count3s_thread() for our third Count 3s solution using private_count array elements.

Figure, Organization of a multi-core computer system on which the experiments are run. Each processor has a private L1 cache; it shares an L2 cache with its "chip-mate" and shares an L3 cache with the other processors.

Multithreaded Solution Three

- How to improve solution three?
  - In the code for third solution, there does not seems to be any shared modified data.
  - The unit of cache coherence is known as a cache line, 64 bytes on this machine.
  - False sharing: logically distinct data shares a physical cache line.

False Sharing



Figure, false sharing. A cache line moves from RAM to the L3 cache, then to the L2 cache, and then to the L1 cache when a thread references its private_count, the copy in the other L1 is invalidated, written back to the L2 cache, and then fetched into the other L1 cache. The line bounces between the L1 cache and the L2 cache, because although the reference are to distinct memory locations, they use the same cache line.

Multithreaded Solution Four

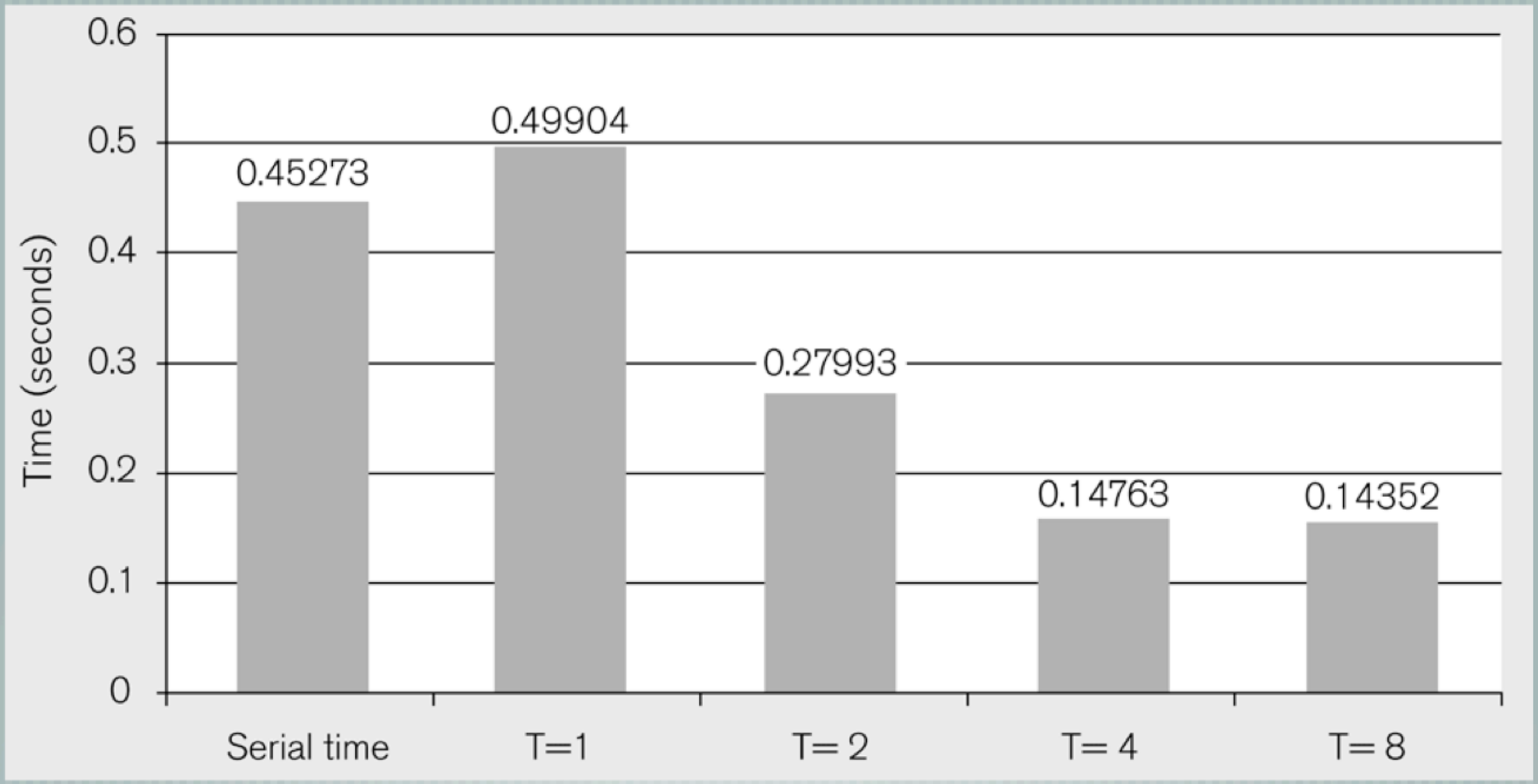
```
struct padded_int
{
  int value;
  char padding[60];
} private_count[MaxThreads];

void count3s_thread(int id)
{
  /*  Compute portion of the array this thread should
      work on */
  int length_per_thread=length/t;
  int start=id*length_per_thread;

  for(i=start; i<start+length_per_thread; i++)
  {
    if(array[i] == 3)
    {
      private_count[id].value ++ ;
    }
  }
  mutex_lock(m);
  count+=private_count[id].value;
  mutex_unlock(m);
}
```

Figure, the count3s_thread() for our fourth Count 3s solution. The private count elements are padded to force them to be allocated to different cache lines.

Multithreaded Solution Four



Figure, performance of our fourth Count 3s solution. Performance is improved as we increase number of threads.

Conclusion

- Parallel programs should have 4 characteristics:
  - correctness,
  - good performance,
  - scalable to large number of processors,
  - portable across many parallel platforms.

Conclusion

- If we want high performance, we can not divorce ourselves entirely from the underlying hardware.
  - Both computational issues and data access can affect performance.
- Lock contention and poor load balancing could hurt performance.

Acknowledgement

- Many pictures used in this talk are from textbook Principles of Parallel Programming by Lin and Snyder.
- Thank you for your attending.
- Any questions?